

串行密码锁实验报告

2023010302 石亦烜
实验时间：2025年5月23日 座号：1

一、实验目的

- 1. 学习使用状态机来控制电路工作，在不同的状态下完成相应的功能。
- 2. 进一步掌握时序逻辑电路的基本分析和设计方法。
- 3. 掌握密码锁的设计原理，用硬件描述语言实现指定功能的密码锁设计。

二、实验内容

设计一个四位 16 进制串行电子密码锁，实现功能：

- 1. 设置密码：用户可以通过按键输入设置密码。
- 2. 验证密码：用户串行输入密码进行验证，密码正确则点亮开启LED灯，否则点亮错误灯。
- 3. 设置管理员密码：可以通过管理员模式设置万能密码。
- 4. 密码锁警报自锁：连续3次开锁失败自锁，只有管理员密码能正确开锁。

三、实验步骤和代码

- 1. 实现代码 通过状态机实现密码锁的功能。状态机包括五种状态，分别记录设置1~4位密码和密码设置完成。源代码password_lock.sv如下：

```
`timescale 1ns / 1ps

module password_lock(
    input wire [3:0] Code,      // 当前输入的4位密码
    input wire Mode,           // 模式选择, 1=设置密码, 0=验证密码
    input wire Manager,        // 管理员模式选择
    input wire CLK,            // 时钟信号
    input wire RST,            // 复位信号, 高跳变复位
    output reg Unlock,          // 解锁信号, 密码正确时输出高电平
    output reg Err,             // 错误信号, 密码错误时输出高电平
    output reg Warning          // 报警信号, 三次失败后触发
);

    reg [15:0] token;           // 用于存储输入的4位密码
    reg [15:0] password;        // 普通密码存储
    reg [15:0] manager_password; // 管理员密码存储
    integer fault_count = 0;    // 密码错误计数

    // 使用枚举类型定义状态
    typedef enum logic [2:0] {
        S0, // 等待输入第1位密码
        S1, // 等待输入第2位密码
        S2, // 等待输入第3位密码
        S3, // 等待输入第4位密码
        S4  // 4位密码输入完成, 等待验证或设置
```

```

} state_t;

state_t state, next_state; // 当前状态和下一个状态

// 状态机主过程：在时钟上升沿或复位时触发
always_ff @(posedge CLK or posedge RST) begin
    if (RST) begin
        state <= S0; // 复位时回到初始状态
    end else begin
        // 根据当前状态，将输入的密码位存入对应的 token 位置
        case (state)
            S0: token[3:0] <= Code; // 存储第1位
            S1: token[7:4] <= Code; // 存储第2位
            S2: token[11:8] <= Code; // 存储第3位
            S3: token[15:12] <= Code; // 存储第4位
            default: ; // 其他状态不操作
        endcase
        state <= next_state; // 状态转移
    end
end

// 状态转移逻辑：根据当前状态决定下一个状态
always_comb begin
    case (state)
        S0: next_state = S1; // 输入第1位后转到输入第2位
        S1: next_state = S2; // 输入第2位后转到输入第3位
        S2: next_state = S3; // 输入第3位后转到输入第4位
        S3: next_state = S4; // 输入第4位后转到验证或设置状态
        S4: next_state = S0; // 完成后回到初始状态
        default: next_state = S0; // 默认回到初始状态
    endcase
end

// 输出逻辑：这里采用时序逻辑，目的是对错误次数进行计数
always_ff @(posedge CLK or posedge RST) begin
    if (RST) begin
        Unlock <= 0;
        Err <= 0;
    end else if (state == S4) begin
        if (Mode && !Warning) begin
            // 设置密码模式
            if (Manager)
                manager_password <= token; // 设置管理员密码
            else
                password <= token; // 设置普通密码
            Unlock <= 0;
            Err <= 0;
        end else begin
            // 验证密码模式
            if (Warning) begin
                // 报警状态下，仅允许管理员密码验证
                if (token == manager_password) begin
                    Unlock <= 1; // 管理员密码正确，解锁并解除报警
                    Warning <= 0;
                end
            end
        end
    end
end

```

```

        fault_count <= 0;
    end else begin
        Unlock <= 0;
        Err <= 1; // 管理员密码错误
        Warning<=1;
    end
end else begin
    // 正常状态下, 验证普通密码或管理员密码
    if ((token == manager_password) || (token == password)) begin
        Unlock <= 1; // 密码正确, 解锁
        fault_count <= 0; // 重置错误计数
    end else begin
        Unlock <= 0;
        Err <= 1; // 密码错误
        fault_count <= fault_count + 1;
        if (fault_count >= 3) begin
            Warning <= 1; // 三次失败后触发报警
        end
    end
end
end
end else begin
    Unlock <= 0;
    Err <= 0;
end
end
endmodule

```

2. 约束文件password_lock.xdc

```

#CLK input
set_property -dict {PACKAGE_PIN J19 IOSTANDARD LVCMOS33} [get_ports CLK];    #CLK
接插孔

#RST input
set_property -dict {PACKAGE_PIN K18 IOSTANDARD LVCMOS33} [get_ports RST];    #IO0
接插孔

# Code input
set_property -dict {PACKAGE_PIN M21 IOSTANDARD LVCMOS33} [get_ports Code[3]];
#IO1接插孔
set_property -dict {PACKAGE_PIN N20 IOSTANDARD LVCMOS33} [get_ports Code[2]];
#IO2接插孔
set_property -dict {PACKAGE_PIN N22 IOSTANDARD LVCMOS33} [get_ports Code[1]];
#IO3接插孔
set_property -dict {PACKAGE_PIN P21 IOSTANDARD LVCMOS33} [get_ports Code[0]];
#IO4接插孔

# Mode and Maager input
set_property -dict {PACKAGE_PIN W21 IOSTANDARD LVCMOS33} [get_ports Mode];
#IO11接插孔

```

```
set_property -dict {PACKAGE_PIN P20 IOSTANDARD LVCMOS33} [get_ports Manager];
#IO10接插孔

# Unlock output
set_property -dict {PACKAGE_PIN AA18 IOSTANDARD LVCMOS33} [get_ports Unlock];
#IO16接插孔

# Err output
set_property -dict {PACKAGE_PIN AB18 IOSTANDARD LVCMOS33} [get_ports Err];
#IO17接插孔
set_property -dict {PACKAGE_PIN AA20 IOSTANDARD LVCMOS33} [get_ports Warning];
#IO18接插孔

# required if touch button used as manual clock source
set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets CLK_IBUF]

set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

四、仿真

在仿真中，我验证了设置普通密码，设置管理员密码，验证普通密码和管理员密码的功能。

1. 仿真代码testbench.sv如下：

```
module testbench();
    reg [3:0] Code;
    reg Mode;
    reg Manager;
    reg clk;
    reg rst;
    wire unlock;
    wire err;
    wire warning;

    reg [15:0] set_code=16'h4321; // 设置的普通密码
    reg [15:0] manager_code=16'h1111; // 设置的管理员密码
    integer i;

    //实例化密码锁模块
    password_lock password_lock_inst(
        .Code(Code),
        .Mode(Mode),
        .Manager(Manager),
        .CLK(clk),
        .RST(rst),
        .Unlock(unlock),
        .Err(err),
        .Warning(warning)
    );
```

```
initial begin
    clk=0;rst=0;
    Mode=1;Manager=0;
    #5 rst=~rst;
    #5 rst=~rst; // 复位信号
    #1 Code=set_code[3:0]; // 输入第1位密码

    #5 clk=~clk;
    #5 clk=~clk;
    #1 Code=set_code[7:4]; // 输入第2位密码
    #5 clk=~clk;
    #5 clk=~clk;
    #1 Code=set_code[11:8]; // 输入第3位密码
    #5 clk=~clk;
    #5 clk=~clk;
    #1 Code=set_code[15:12]; // 输入第4位密码
    #5 clk=~clk;
    #5 clk=~clk;
    #5 clk=~clk;
    #5 clk=~clk;

    #20 Manager=1; // 设置管理员密码模式
    #5 rst=~rst;
    #5 rst=~rst; // 复位信号
    #1 Code=manager_code[3:0]; // 输入第1位密码
    #5 clk=~clk;
    #5 clk=~clk;
    #1 Code=manager_code[7:4]; // 输入第2位密码
    #5 clk=~clk;
    #5 clk=~clk;
    #1 Code=manager_code[11:8]; // 输入第3位密码
    #5 clk=~clk;
    #5 clk=~clk;
    #1 Code=manager_code[15:12]; // 输入第4位密码
    #5 clk=~clk;
    #5 clk=~clk;
    #5 clk=~clk;
    #5 clk=~clk;
    Mode=0;Manager=0;

    //验证密码,正确密码1234
    #5 rst=~rst;
    #5 rst=~rst; // 复位信号
    for(i=1;i<5;i++)begin
        #1 Code=i;
        #5 clk=~clk;
        #5 clk=~clk;
    end
    #5 clk=~clk;
    #5 clk=~clk;

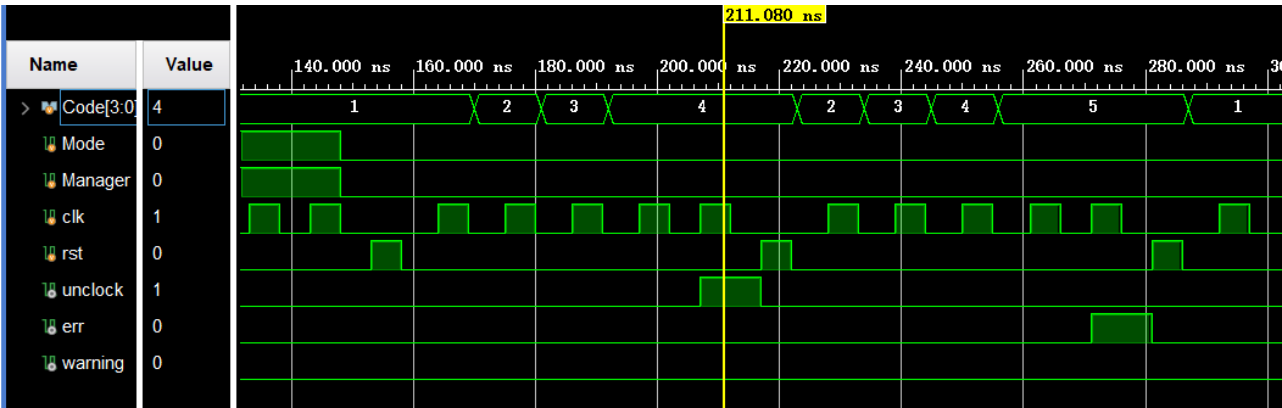
    //验证密码,错误密码2345
    #5 rst=~rst;
    #5 rst=~rst; // 复位信号
```

```
for(i=2;i<6;i++)begin
    #1 Code=i;
    #5 clk=~clk;
    #5 clk=~clk;
end
#5 clk=~clk;
#5 clk=~clk;

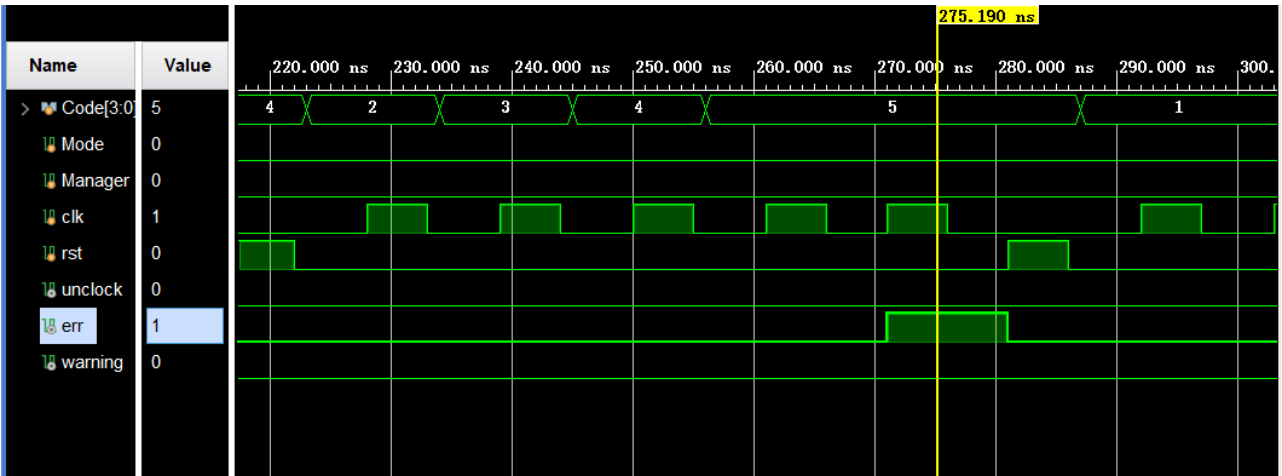
//验证管理员密码,1111
#5 rst=~rst;
#5 rst=~rst; // 复位信号
for(i=1;i<=4;i++)begin
    #1 Code=1;
    #5 clk=~clk;
    #5 clk=~clk;
end
#5 clk=~clk;
#5 clk=~clk;
end

endmodule
```

2. 仿真结果如下:



上图中可以看出，在输入正确的普通密码1234后，解锁信号unlock正确输出



输入错误密码2345后，错误信号err输出。



输入正确的管理员密码1111，解锁信号unlock正确输出。

五、小结

此次实验通过对一个四位串行密码锁的多种功能的实现，我熟悉了利用状态机的设计过程，加深了对时序逻辑电路设计方法的掌握。

在此次试验的过程中，我遇到的最大的问题就是如何完成多次错误后警报自锁的功能。起初我采用的是组合逻辑来完成状态机的输出部分，并在组合逻辑中使用了`error_times++`这样的语句意图实现错误计数，但在实际运行后我明白了这样的语句在组合逻辑中会导致变量值一直不停的叠加。随后我改用时序逻辑实现状态输出模块才正确解决了这个问题。通过对这个问题的解决，加深了我对system verilog代码块实际运行过程的理解，更清晰的区分了组合逻辑和时序逻辑的不同功能。